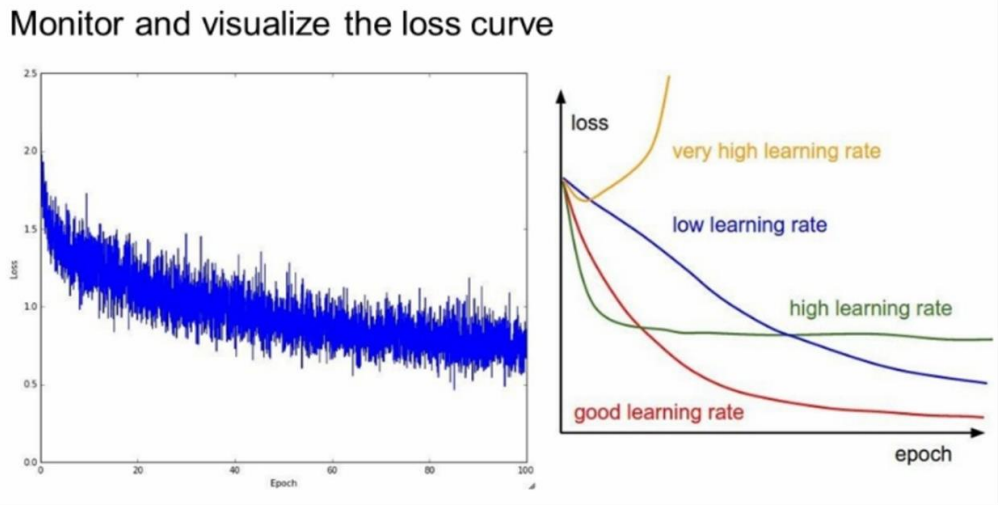
Optimizer

미래연구소 16기 7주차

0. Learning Rate 1) learning rate에 따른 train loss 변화



1> ‘good learning rate’ 처럼 지속적으로 감소하는 형태가 가장 이상적입니다.

2> 이에 따라 learning rate를 조정하면 됩니다.

3> 그래도 학습이 잘 안 되는 경우 initialization의 변경 혹은 optimizer를 변경해봅니다.

출처: CS231n 2017 winter

1. SGD 1) 정의

1> random하게 추출한 mini-batch씩 gradient descent 진행한다.

2> (in tensorflow)

batch_size = mini-batch size (default =None = 32)

fit

```
fit(x=None, y=None, batch_size=None, epochs=1, verbose=1, callbacks=None,
```

- **batch_size**: Integer or `None`. Number of samples per gradient update. If unspecified, `batch_size` will default to 32. Do not specify the `batch_size` if your data is in the form of symbolic tensors, generators, or `Sequence` instances (since they generate batches).

1. SGD 2) 단점

해당 위치에서의 gradient만 고려한다.

<문제점>

1> gradient가 dimension 별로 차이가 크면 minimum을 찾기 어렵다.

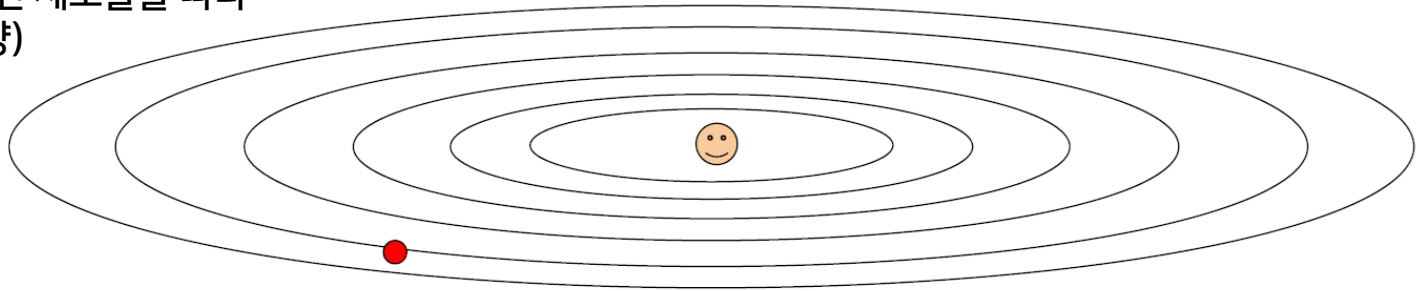
2> local minimum (분지?) 혹은 plateaus (골짜기 혹은 고원)
(즉, 일시적으로 gradient가 0에 가까운 지점)
=> gradient descent 멈춘다.

무슨 말?

문제 1> Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?

(현재 모양 상상되시나요?
A4용지 중앙의 긴 세로줄을 따라
반으로 접은 모양)



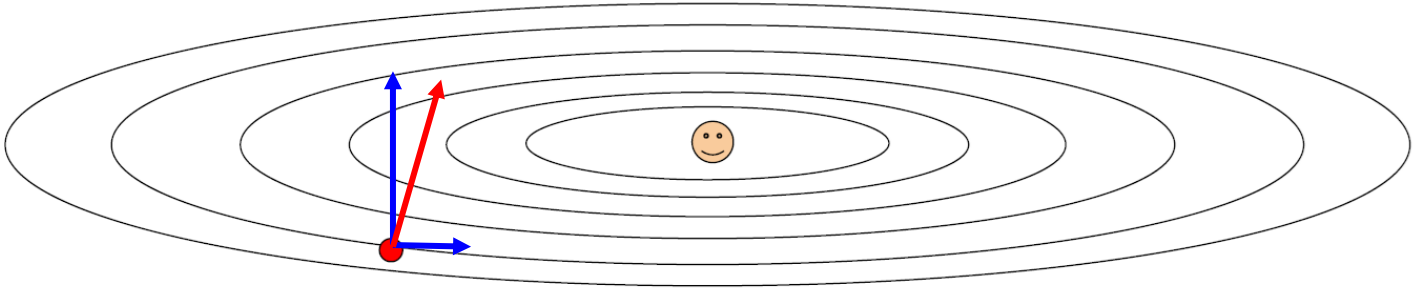
Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

1> 수평 방향으로는 기울기가 완만하다 => gradient 작다.

2> 수직 방향으로는 기울기가 가파르다 => gradient 크다.

문제 1> Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?



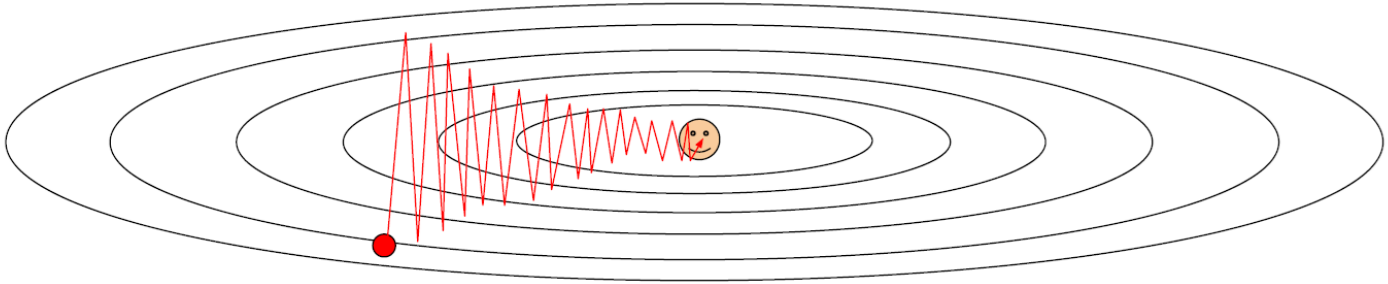
Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

이게 왜 문제일까?

해당 점만 고려하는 것이 아니라 전체적인 방향성을 고려해야 minimum에 도달 가능

문제 1> Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction

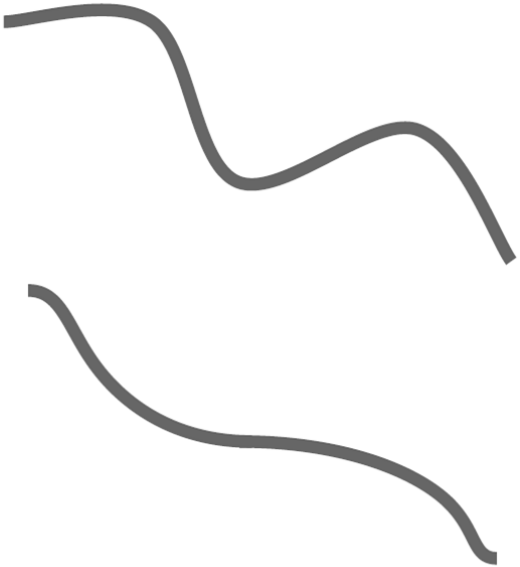


Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

이렇게 학습에 많은 시간이 필요하다.

문제 2> Optimization: Problems with SGD

What if the loss function has a **local minima** or **saddle point**?

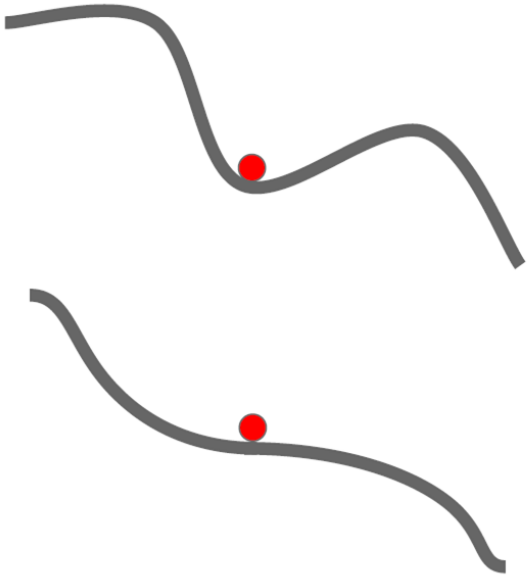


이런 구간에서 학습이 잘 될까?

문제 2> Optimization: Problems with SGD

What if the loss function has a **local minima** or **saddle point**?

Zero gradient, gradient descent gets stuck



저런 골짜기에 갇히면 못 빠져 나온다.

하지만 지구에서 구슬을 굴리는 상황을 생각해보면 저정도 골짜기는 극복가능
=> why?

1. SGD 3) 해결책: 관성

구슬처럼 gradient descent도 관성을 주면 좋을 것 같다.
(관성 = 이전 운동 상태를 지속하려는 성질) == (이전 gradient도 GD에 반영하자)

1> 이전의 움직임까지 고려하니 1번 문제점 해결

2> 관성이 가해지면 당연히 local minimum 혹은 plateaus는 극복 가능하다.

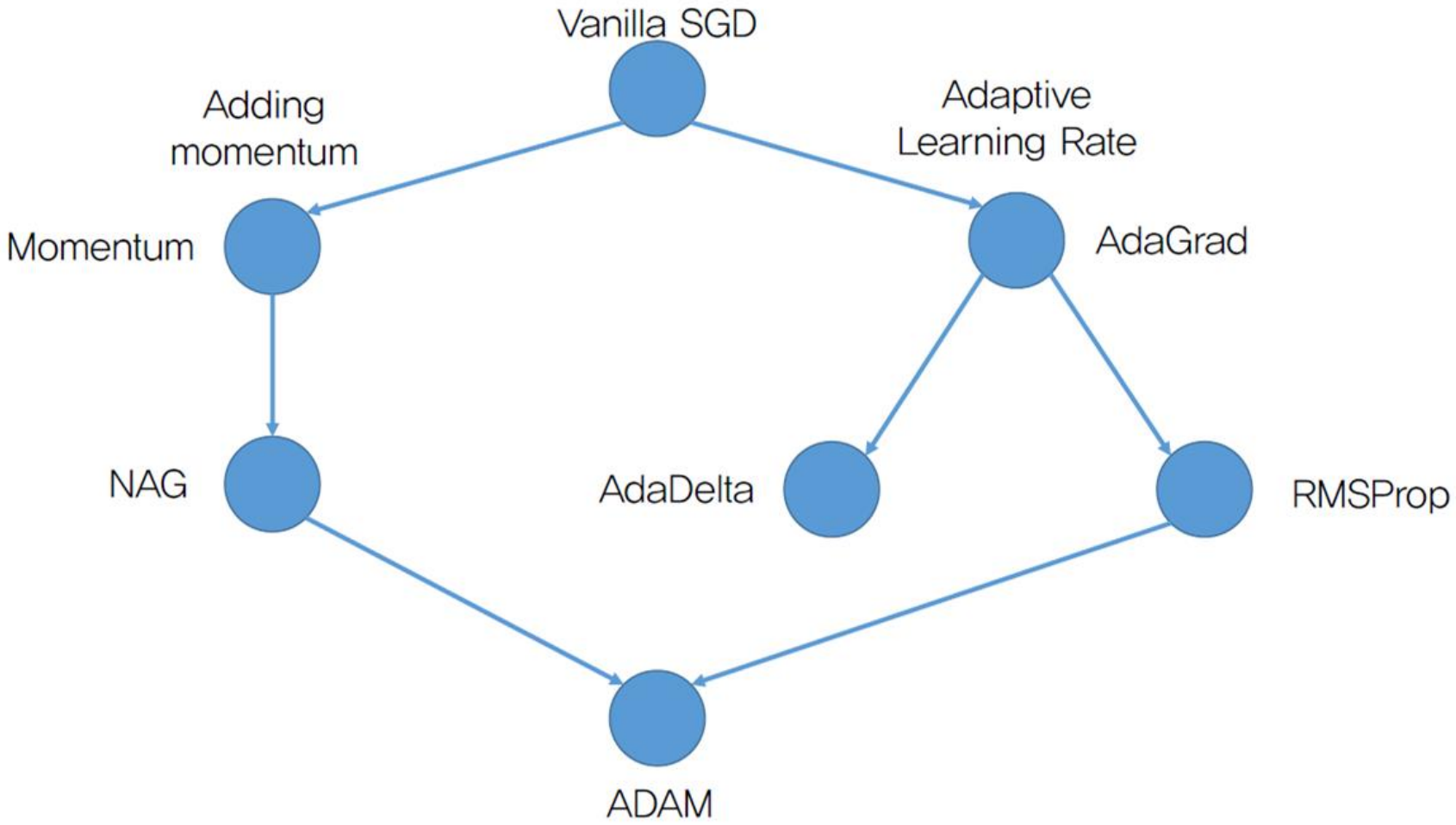
1. SGD 3) 해결책: 관성

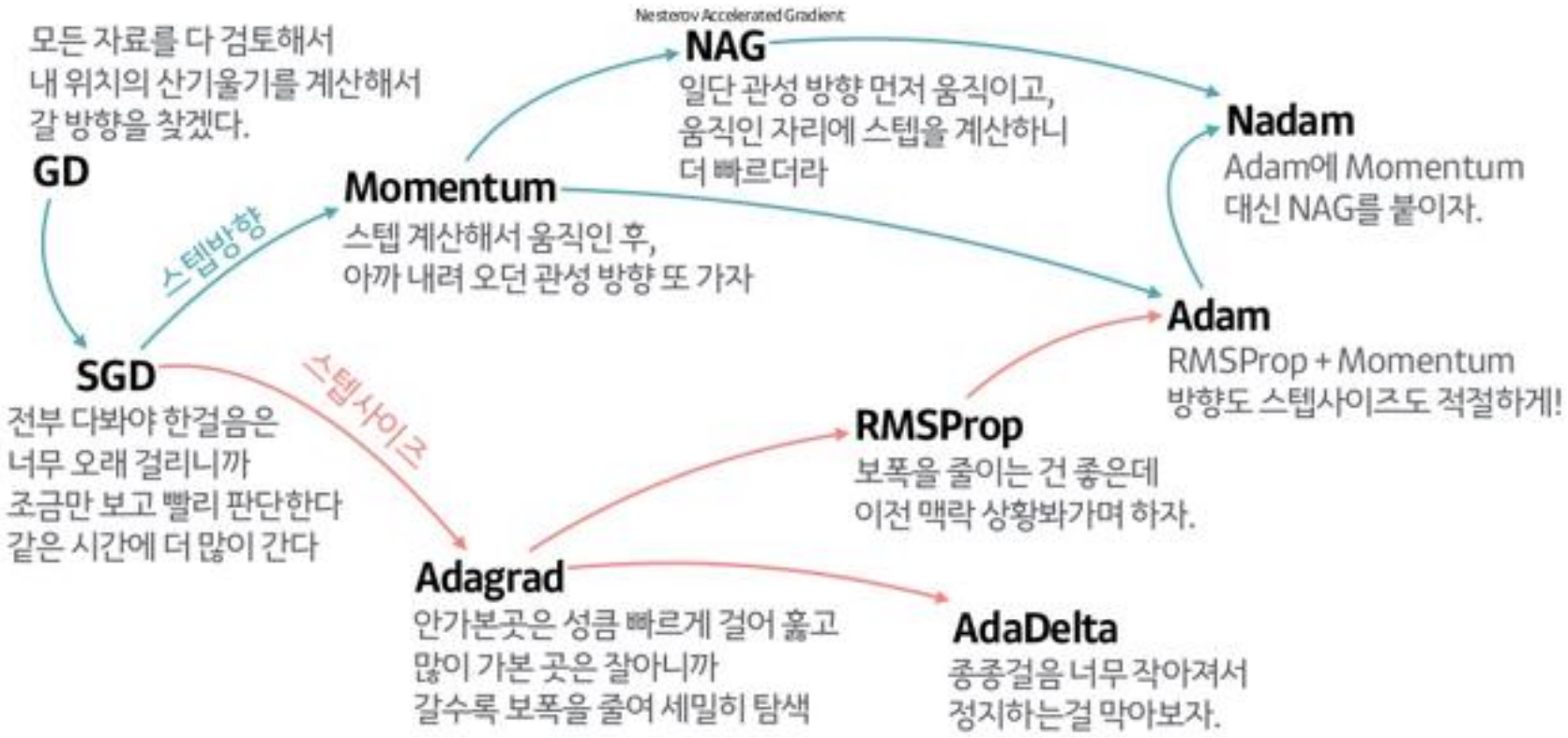
3> vector = 방향, 크기를 모두 갖는 물리량
방향 = gradient
크기 = learning rate

$$W := W - \alpha \frac{\partial}{\partial W} cost(W)$$

4> 그래서 optimizer에는 2가지 계보가 있습니다.

gradient에 관성을 주는 경우
learning rate에 관성을 주는 경우


Vanilla SGD
Adding momentum
Adaptive Learning Rate
Momentum
AdaGrad
NAG
AdaDelta
RMSProp
ADAM


모든 자료를 다 검토해서
내 위치의 산기울기를 계산해서
갈 방향을 찾겠다.
GD
스텝방향
SGD
전부 다봐야 한걸음은
너무 오래 걸리니까
조금만 보고 빨리 판단한다
같은 시간에 더 많이 간다
Momentum
스텝 계산해서 움직인 후,
아까 내려 오던 관성 방향 또 가자
Nesterov Accelerated Gradient
NAG
일단 관성 방향 먼저 움직이고,
움직인 자리에 스텝을 계산하니
더 빠르더라
Nadam
Adam에 Momentum
대신 NAG를 붙이자.
Adam
RMSProp + Momentum
방향도 스텝사이즈도 적절하게!
스텝사이즈
RMSProp
보폭을 줄이는 건 좋은데
이전 맥락 상황봐가며 하자.
Adagrad
안가본곳은 성큼 빠르게 걸어 훑고
많이 가본 곳은 잘아니까
갈수록 보폭을 줄여 세밀히 탐색
AdaDelta
종종걸음 너무 작아져서
정지하는걸 막아보자.

식은 중요하지 않습니다.

밑의 설명 듣고 다시 이전 슬라이드 계보를 보면
각 기법의 느낌이 이해됩니다.

각 기법의 특징만 알면 됩니다.

2. SGD + Momentum 1) 식

$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$$

1> v_(t-1)에 이전 속도(방향)가 저장되어 있다.

2> γ를 곱해 이전 속도를 얼마나 반영할 지 결정

<기존에 배운 용어>
1. θ = w (parameter)
2. η(eta) = α (learning rate)
3. ▽_θJ(θ) = dW (gradient)

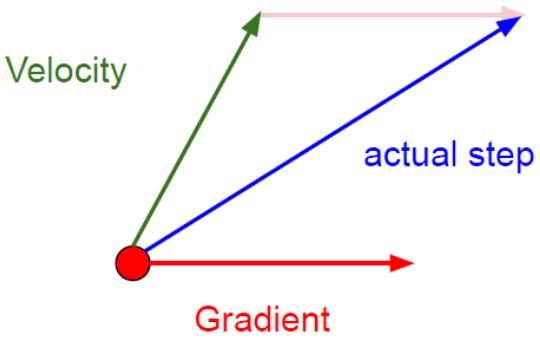
<처음 등장한 용어>
1. γ = momentum
2. v_t = t의 속도
(현재의 움직이는 방향)

2. SGD + Momentum 2) 의미

$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$$

Momentum update:



gradient (dW) 방향으로 가야하는데

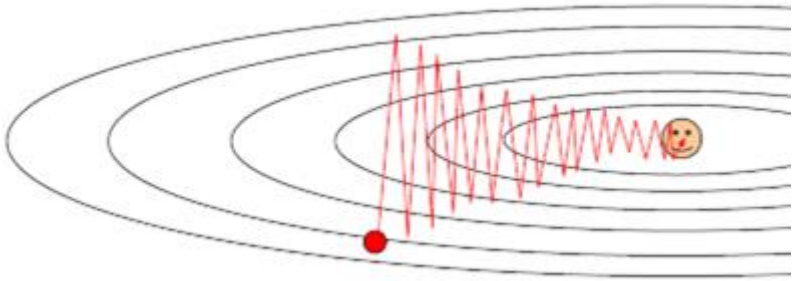
velocity (v_(t-1))의 영향을 받아

actual step (momentum 방식의 gradient) 으로 간다.

2. SGD + Momentum 2) 의미

$$\theta = \theta - v_t$$

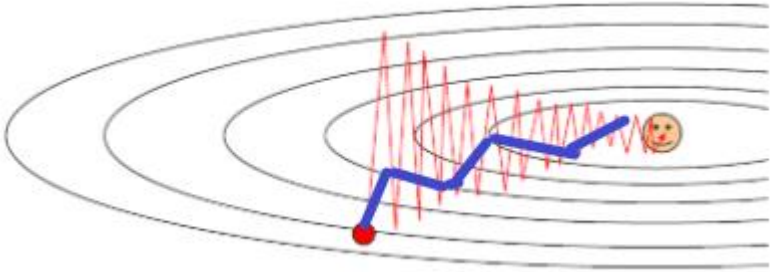
$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$$



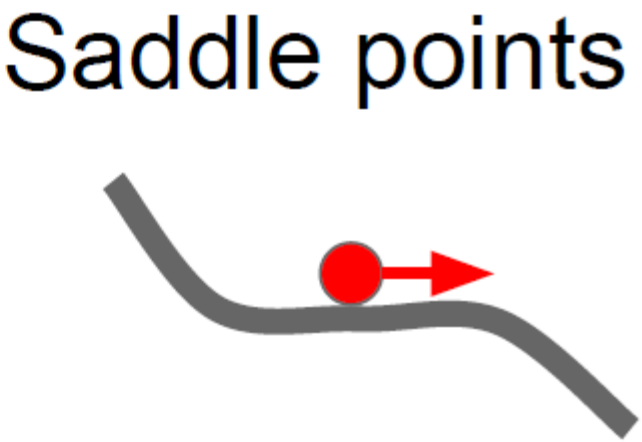
(수직 방향 생각)
1> dW는 위의 방향

2> v_(t-1)가 반대방향

3> 이들이 더해지니 값이 상쇄

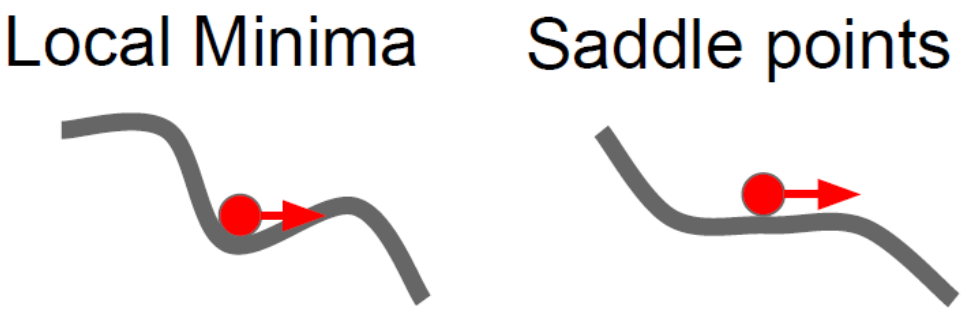


2. SGD + Momentum 2) 의미



이것도 빠져나올 수 있다.

2. SGD + Momentum 3) 문제점



하지만 global minimum도 빠져나오면? 문제다.

(앞을 보지 않아 생긴 일)

앞을 보고 걷자

2. SGD + Momentum 4) hyperparameter

$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$$

1> [0.5, 0.9, 0.95, 0.99] 중에 보통 0.9를 사용

2> γ가 클수록 이전 속도의 영향력이 커진다.
(local minimum에서 잘 빠져나감 vs 브레이크를 잘 못 잡는 case)

3. NAG (Nesterov Accelerated Gradient) 1) 식

$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta - \gamma v_{t-1})$$

1> ▽_θJ(θ-momentum)

momentum 방향으로 갔을 때 예상되는 지점의 gradient를 사용한다.

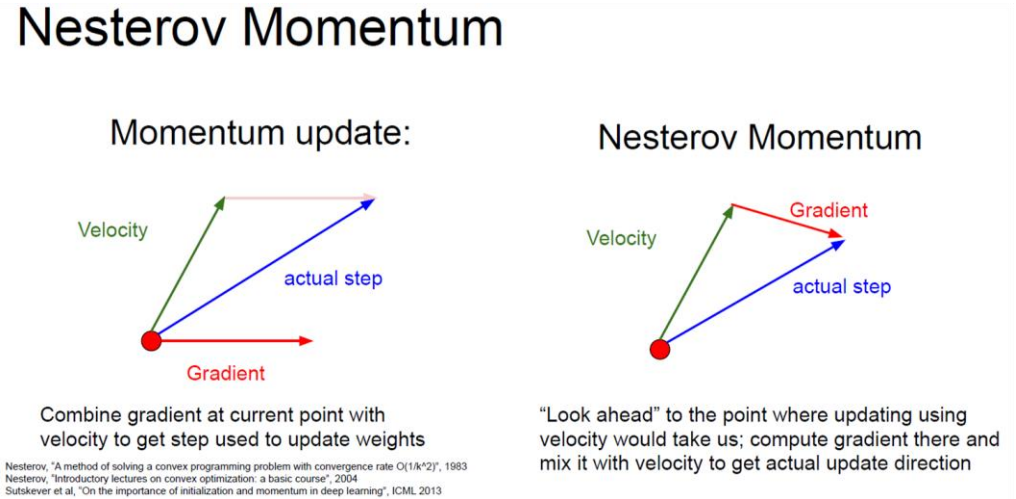
1. θ = w (parameter)
2. η(eta) = α (learning rate)
3. ▽_θJ(θ) = dW (gradient)

1. γ = momentum
2. v_t = t의 속도
(현재의 움직이는 방향)

3. NAG (Nesterov Accelerated Gradient) 2) 의미

$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta - \gamma v_{t-1})$$



1> momentum 방향으로 갔을 때 예상되는 지점의 gradient를 사용한다.

2> 그래서 더 converge 잘 되고

3> minimum도 더 잘 찾음

지금까지 gradient 계보였습니다.
이제는 learning rate (step size)계보를 봅니다.

4. Adagrad (Adaptive Gradient) 1) 식

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = G_{t-1} + (\nabla_\theta J(\theta_t))^2$$

1> G_(t-1)에 이전 속력이 저장되어 있다.

2> 누적된 속력으로 learning rate를 나눈다.

<기존에 배운 용어>
1. θ = w (parameter)
2. η(eta) = α (learning rate)
3. ▽_θJ(θ) = dW (gradient)

<처음 등장한 용어>
1. G_t = t의 속력
(현재의 움직임)
2. ε = G_t가 0인 상황 방지

4. Adagrad (Adaptive Gradient) 2) 의미

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = G_{t-1} + (\nabla_\theta J(\theta_t))^2$$

1> G_(t-1)에 저장된 이전 속력이 계속 빨랐다면
learning rate를 나눠서 속력을 늦춘다.

2> G_(t-1)에 저장된 이전 속력이 계속 느렸다면
learning rate를 나눠서 속력을 가속화한다.

4. Adagrad (Adaptive Gradient) 3) 문제점

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = G_{t-1} + (\nabla_\theta J(\theta_t))^2$$

1> 속력이 계속해서 증가하면
learning rate를 너무 큰 수로 나눠서 학습이 안 될 수 있다.

2> 속력이 초반에만 크고 나중에는 작으면 굳이 속력을 줄이지 않아도 되는데 줄이게 된다.

=> 인접한 속력에 가중치를 더 준다. (마치 sgd에서 momentum을 주는 형태)

4. Adagrad (Adaptive Gradient) 4) hyperparameter

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = G_{t-1} + (\nabla_\theta J(\theta_t))^2$$

1> ε는 [1e-4, 1e-5, 1e-6, 1e-7, 1e-8] 중에서 보통 1e-8을 사용한다.

5. Rmsprop (Root Mean Squared Prop) 1) 식

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = \gamma G_{t-1} + (1-\gamma)(\nabla_\theta J(\theta_t))^2$$

1> G_(t-1)에 이전 속력이 저장되어 있다.

2> 인접한 속력에 더 가중치를 가해서
learning rate를 나눈다.

<기존에 배운 용어>
1. θ = w (parameter)
2. η(eta) = α (learning rate)
3. ▽_θJ(θ) = dW (gradient)

<처음 등장한 용어>
1. G_t = t의 속력
(현재의 움직임)
2. ε = G_t가 0인 상황 방지

5. Rmsprop (Root Mean Squared Prop) 2) 의미

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = \gamma G_{t-1} + (1-\gamma)(\nabla_\theta J(\theta_t))^2$$

1> 인접한 속력의 영향력을 더 받아서
이전의 속력이 큰 것 때문에 step size가 0에 가까워 지는 걸 방지

5. Rmsprop (Root Mean Squared Prop) 3) hyperparameter

$$\theta_{t+1} = \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_\theta J(\theta_t)$$

$$G_t = \gamma G_{t-1} + (1-\gamma)(\nabla_\theta J(\theta_t))^2$$

1> ε는 [1e-4, 1e-5, 1e-6, 1e-7, 1e-8] 중에서 보통 1e-8을 사용한다.

2> γ는 보통 0.9를 사용한다.

gradient 계보와 learning rate 계보를 합친다면
더 좋은 성능을 가질 것

6. Adam (Adaptive Moment Estimation) 1) 식

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

6. Adam (Adaptive Moment Estimation) 2) 의미

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

1> step size와 step direction 모두 관성을 부여했다.

2> 대체적으로 성능이 좋다.

6. Adam (Adaptive Moment Estimation) 3) hyperparameter

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

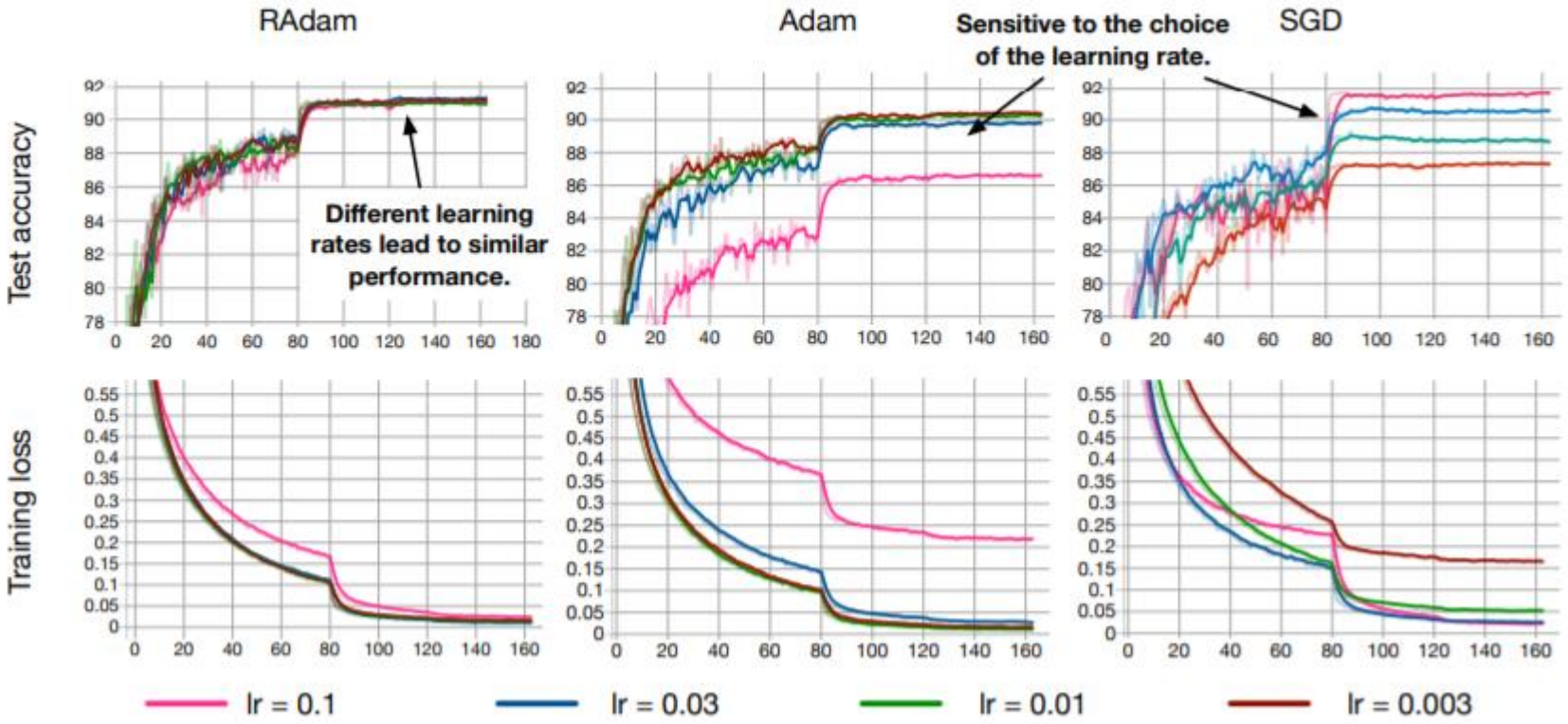
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

1> 보통
β_1 = 0.9
β_2 = 0.999
ε = 1e-8

을 주로 사용한다.

2019년 8월 Adam을 더 보완한
Rectified Adam이 나옴

Adam 계열의 경우
학습이 진행될수록 learning rate가 작아져서
bad learning rate에 converge할 수 있는데
이 감소를 rectified하게 만드는 방법



논문: https://arxiv.org/pdf/1908.03265.pdf
논문 해설: https://flonelin.wordpress.com/2019/09/12/radamon-the-variance-of-the-adaptive-learning-rate-and-beyond/
사용해보고 싶은 분(Radam 공식 github): https://github.com/CyberZHG/keras-radam

6기에서 optimizer 여러가지 시도 해본 사례

왜 보여드렸을까요?

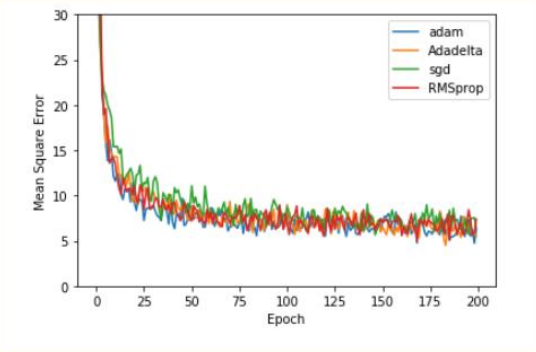
CASE.2

```
[18] #optimizers 비교 그래프
     opti_name = ["adam","Adadelta","sgd","RMSprop"]
     for name in opti_name:
       model = Sequential()
       model.add(BatchNormalization())
       model.add(Dense(units = 128, input_dim = 128, activation = 'relu'))
       model.add(BatchNormalization())
       model.add(Dense(units = 128, activation = 'relu'))
       model.add(BatchNormalization())
       model.add(Dense(units = 128, activation = 'relu'))
       model.add(BatchNormalization())
       model.add(Dense(1))
       model.compile(loss = "mse", optimizer= name, metrics = ['mse'])
       hist = model.fit(xdata, ydata, epochs = 200)
       print(hist.history['loss'])
       history = pd.DataFrame(hist.history)
       history['epoch'] = hist.epoch
       plt.plot(history['epoch'], history['mean_squared_error'],label = name)
     plt.xlabel('Epoch')
     plt.ylabel('Mean Square Error')
     plt.ylim([0,30])
     plt.legend()
     plt.show()
```

-**INPUT** 정규화를 위해 각 층마다 **BATCH NORMAL**을 실행

-활성화 함수는 **RELU**로, **LOSS**는 **MSE**로 고정시킨 후 **OPTIMIZER**를 비교

CASE.2



-**OPTIMIZER**는
Adam
Adadelta
SGD
RMSprop 로 실행

-약간의 차이만 보임

<최종 결론>

대체적으로 adam이 좋지만 항상 그렇지는 않습니다.
그래서 시간적 여유가 되면 가능한 다 사용해서 그래프 그려보고
가장 잘 되는 걸 쓰는 것이 좋습니다.

(계속해서 어떤 data든 잘 학습할 수 있는 optimizer를 만들고 있지만
아직 그런 optimizer는 없습니다.)
(다행인 건 optimizer에서는 lr을 제외하고
다른 hyperparameter를 tuning에 공을 들일 필요는 없습니다.)